

Sambi Kopi POS

Code Explanation Documentation

Purpose

This document explains important code concepts that may be asked during presentation: JavaFX entry point, core inheritance, overriding, overloading, database, MVC, data structures, TableView, Chart, and presentation questions.

Stack

Java, JavaFX, FXML, Maven, SQLite

Roles

Barista, Cashier, Owner

Core OOP

Inheritance, Overriding, Overloading

Table of Contents

1

Main Entry Point

App.java and JavaFX start method

2

Core Inheritance

CafeItem parent model

3

Inheritance Example

CafeMenuItem and InventoryItem

4

Overriding Example

getItemType() in child classes

5

Overloading Example

addMenuForReview(...) versions

6

Encapsulation

Private fields and accessors

7

Abstraction

DatabaseManager hides SQL setup

8

Composition

Menu ingredients from stock

9

MVC Pattern

Model, View, Controller

10

Database

SQLite tables and schema

11

TableView

Database data display

12

Chart

Owner report visualization

13

Data Structures

ObservableList, FilteredList, List

14

Possible Questions

Presentation Q&A;

1 Main Entry Point

The main entry point is:

```
src/main/java/interva/sambikopi/App.java
```

App.java extends JavaFX Application:

```
public class App extends Application
```

This is required by JavaFX. The start(Stage stage) method is called when the application starts.

2 Core Inheritance

Inheritance is applied in the core model layer, not only in JavaFX.

Parent class

```
src/main/java/interva/sambikopi/model/CafeItem.java
```

Child classes

```
src/main/java/interva/sambikopi/model/CafeMenuItem.java
src/main/java/interva/sambikopi/model/InventoryItem.java
```

Relationship

```
CafeItem |-- CafeMenuItem |-- InventoryItem
```

CafeItem contains common item concepts such as item name and status. CafeMenuItem represents menu data, while InventoryItem represents stock data.

3 Inheritance Example

```
public class CafeMenuItem extends CafeItem
public class InventoryItem extends CafeItem
```

This means CafeMenuItem and InventoryItem inherit common behavior from CafeItem.

4 Overriding Example

Both child classes override methods from CafeItem. Each child class gives its own implementation.

Example in CafeMenuItem

```
@Override public String getItemType() {
    return "Menu Item";
}
```

Example in InventoryItem

```
@Override public String getItemType() { return
    "Inventory Item";
}
```

5 Overloading Example

Method overloading is used in:

```
src/main/java/interva/sambikopi/model/SambiKopiDataStore.java
```

Both methods have the same name, but different parameters:

```
public static void addMenuForReview(CafeMenuItem item)
public static void addMenuForReview(CafeMenuItem item, List<MenuIngredient> ingredients)
```

This is method overloading because the method name is the same but the parameter list is different.

6 Encapsulation

Encapsulation is applied in model classes. Fields are private and accessed through getters and setters.

```
private String product;
public String getProduct() {
    return product;
}
public void setProduct(String product) {
    this.product = product;
}
```

7 Abstraction

Abstraction is applied in DatabaseManager.java. Controllers do not need to manually create database tables or open SQLite connections. They use database helper methods instead.

```
DatabaseManager.initializeDatabase();
DatabaseManager.getConnection();
```

The SQL setup is hidden inside DatabaseManager.

8 Composition

Composition is used in menu ingredients. A menu item can be composed of several stock ingredients.

```
Matcha Latte
- Matcha Powder x1 scoop
- Fresh Milk x1 bottle
```

This relationship is stored in the menu_ingredients database table.

9 MVC Pattern

Model	View	Controller
• Cafeltem • CafeMenuitem • InventoryItem • OrderItem • MenuIngredient	• menu.fxml • menu_list.fxml • stock.fxml • orders.fxml • cashier_menu.fxml • owner_report.fxml	• MainController.java • MenuController.java • MenuListController.java • StockController.java • OrdersController.java • CashierMenuController.java

Controllers connect user actions with model and database logic.

10 Database

The project uses SQLite.

Database file

```
sambi_kopi.db
```

Schema file

```
database_schema.sql
```

Important tables

- menu_items
- inventory_items
- menu_ingredients
- orders
- app_settings

11 TableView

TableView is used to display database data. Examples:

- Menu List table
- Stock table
- Orders table
- Owner Review Stock table
- Owner Review New Menu table

12 Chart

Owner Report uses charts to visualize report data, such as order status and payment methods.

13 Data Structures

The project uses:

- ObservableList
- List
- FilteredList
- ArrayList

These are used for storing and displaying menu, stock, order, and ingredient data.

14 Possible Questions

Q: Where is inheritance used?

A: In the model layer. CafeMenuItem and InventoryItem inherit from CafeItem.

Q: Where is overriding used?

A: In CafeMenuItem and InventoryItem, especially in getItemType().

Q: Where is overloading used?

A: In SambiKopiDataStore.java, where addMenuForReview(...) has two versions with different parameters.

Q: Why is App extends Application not enough?

A: It is inheritance, but it comes from the JavaFX framework. The project also applies inheritance in core model code through CafeItem.

Q: What happens when an order is completed?

A: The order status changes to Complete and the linked stock ingredients are reduced.

Q: Why does the Cashier menu only show approved menus?

A: New menu items must be approved by Owner before they can be sold.

Q: Why store image path instead of image binary in the database?

A: It keeps the database smaller and easier to manage.

15 Quick OOP Summary

Concept	Example in Project
Inheritance	CafeMenuItem and InventoryItem extend CafeItem
Overriding	getItemType() returns different values in each child class
Overloading	addMenuForReview(...) with different parameters
Encapsulation	Private fields with getters and setters in model classes
Abstraction	DatabaseManager hides SQLite setup
Composition	Menu items are composed of ingredients
MVC	Model classes, FXML views, and controller classes